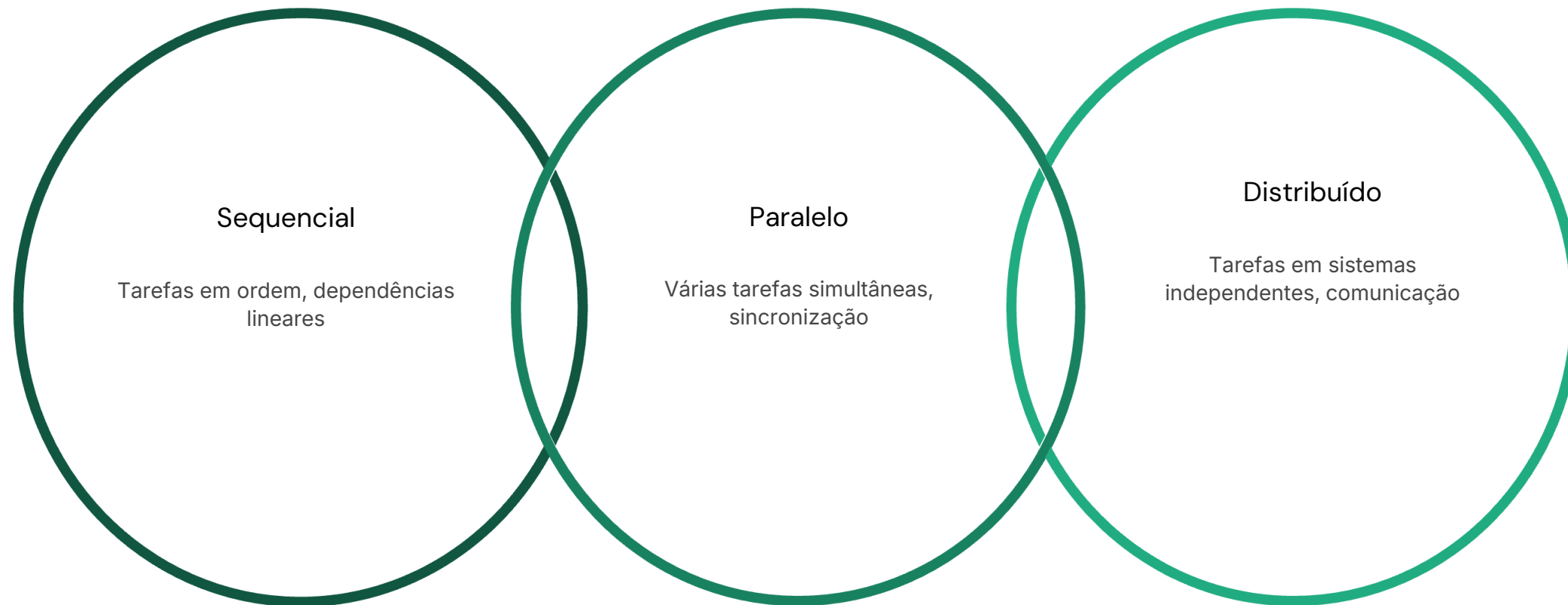


Implementação Sequencial, Paralela e Distribuída da Equação de Difusão de Calor 2D



Filipe Antonio de Lima Nogueira

João Victor da Cruz Silvestre

Matheus Henrique Consoni
Mazanti

Contexto e Motivação

Difusão de calor = problema clássico em engenharia

Alto poder computacional

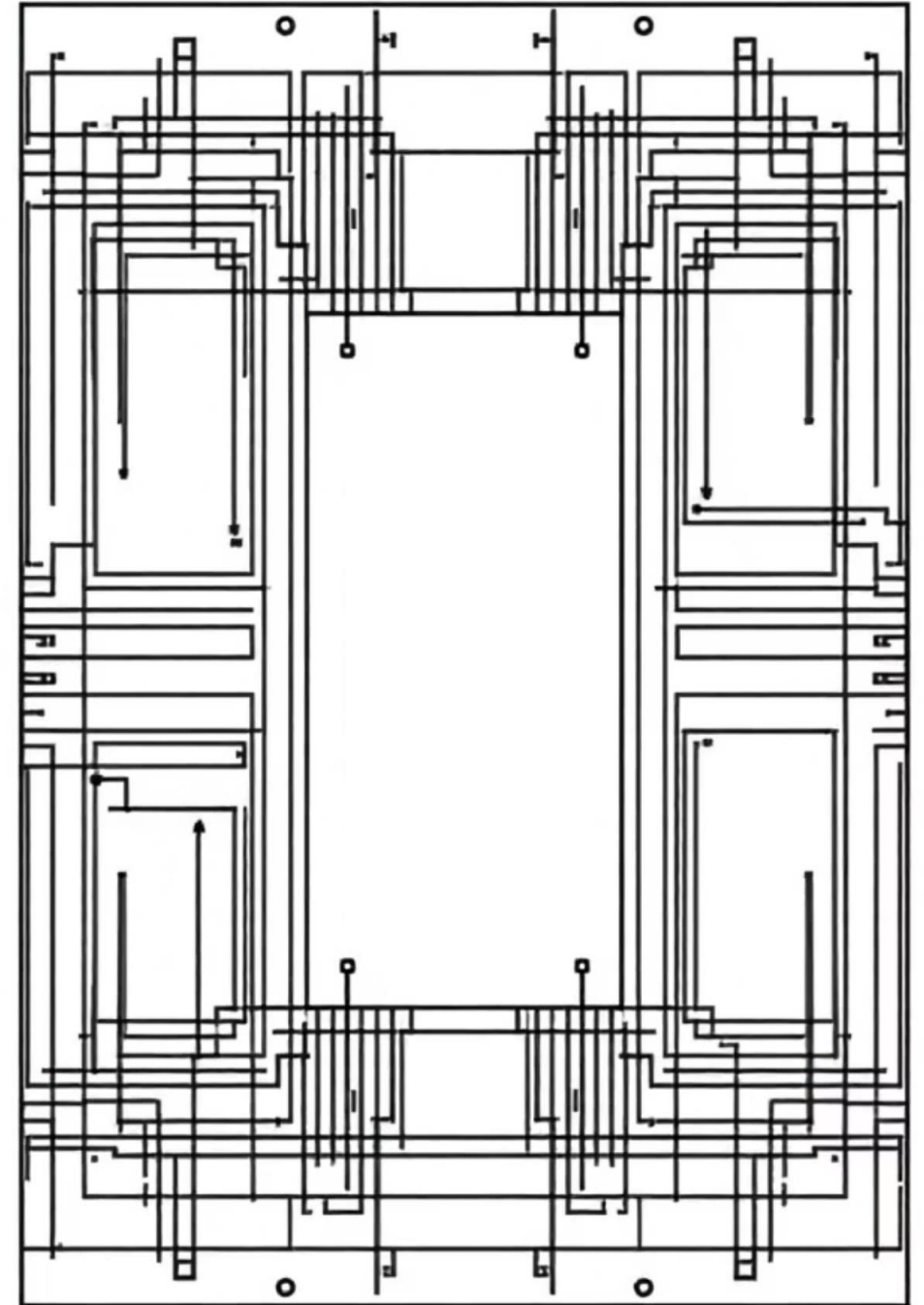
Requer alto poder computacional

Aplicações diversas

Simulações usadas em diversos setores (indústria, engenharia, refrigeração, etc.)

Problema Modelado

- Placa 2D
- Condições de contorno estáticas
- Equação de difusão envolvendo a variação temporal da temperatura



Método Numérico Utilizado



Diferenças Finitas

Método numérico para
discretização espacial



Esquema FTCS

Forward Time Central Space



Critério de estabilidade CFL

Garantia de convergência
numérica

Objetivos do Projeto

1. Implementar as versões:

- Sequencial
- Paralela (threads Python)
- Distribuída (sockets TCP/IP)

2. Análise e Avaliação

- Realizar benchmarks
- Avaliar desempenho, speedup e eficiência
- Identificar limitações
- Propor melhorias

Ambiente de Desenvolvimento



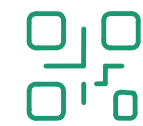
Hardware

Notebook i7, 16GB RAM



Linguagem

Python 3.10



Bibliotecas

- NumPy
- Matplotlib
- Pandas

Configuração da Versão Distribuída



Comunicação TCP/IP



Sockets



Serialização com Pickle



Parâmetros de Teste



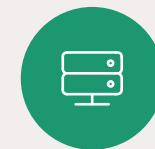
Tamanhos

20×20, 50×50, 100×100



Threads

1–8



Workers

1–4



α

0,01



Convergência

1e-4



Condições de Contorno

Bordas:

- Esquerda, direita e topo = 0°C
- Base = 100°C

Interior = 25°C

Fonte térmica central = 80°C

Versão Sequencial

01

Implementação direta

02

Duplo loop

03

Usada como baseline

04

Complexidade $O(N^2)$

 **Baseline de referência** para
comparação com implementações
paralelas e distribuídas

Versão Paralela (Threads)



Divisão Horizontal da Malha

A malha de dados é particionada horizontalmente, permitindo que diferentes threads processem sub-regiões simultaneamente para otimizar o desempenho.



Sincronização de Threads

Threads utilizam barreiras para garantir que todas atinjam um ponto comum antes de avançarem, mantendo a consistência e a integridade dos dados.



Desafios da Implementação

A arquitetura de threads em Python apresenta obstáculos como o Global Interpreter Lock (GIL) e o custo computacional da sincronização.

GIL (Global Interpreter Lock)

Um mecanismo do CPython que permite apenas uma thread executar bytecode Python por vez, limitando o paralelismo real para tarefas intensivas em CPU.

Overhead de Sincronização

O custo de desempenho associado à coordenação das threads, como a espera por barreiras, pode reduzir os ganhos de performance do paralelismo.

Versão Distribuída



Arquitetura Master-Worker

Um nó central ('Master') coordena tarefas e distribui partes da malha de dados para múltiplos nós trabalhadores ('Workers'), gerenciando a carga de trabalho.



Troca de Ghost-Rows

Workers trocam dados de borda ('ghost-rows') com vizinhos adjacentes para garantir a consistência dos cálculos nas fronteiras da malha.



Particionamento da Malha

Cada Worker recebe e processa uma seção distinta da malha de dados, permitindo processamento paralelo e escalabilidade da computação.



Comunicação via Socket

A comunicação eficiente entre Workers, incluindo o envio de ghost-rows, é realizada através de sockets para troca rápida de dados.

Tempos de Execução



Versão Sequencial

A linha de base de desempenho, sem otimizações. O tempo de execução aumenta linearmente com o tamanho da malha, representando a abordagem padrão.



Versão Paralela

Apresenta melhorias significativas no desempenho, especialmente notáveis em malhas de tamanho médio a grande, devido à utilização eficiente de múltiplos núcleos de CPU.



Versão Distribuída

Introduz um *overhead* inicial que a torna menos eficiente para malhas pequenas. Contudo, torna-se a opção mais vantajosa para malhas muito grandes, onde a escalabilidade é crucial.

Speedup – Versão Paralela

Ganho Máximo

Até **2,34x** com 8 threads, demonstrando a eficiência da paralelização.

Escalabilidade

Os melhores ganhos são observados com malhas computacionais de maior tamanho.

Speedup – Versão Distribuída

Overhead em Malhas Pequenas

Speedup negativo em 20×20 , demonstrando o custo da comunicação e sincronização em tarefas pequenas.

Ganho em Malhas Grandes

Melhora significativa para 100×100 com múltiplos workers, indicando maior eficiência em problemas complexos.

Eficiência

Versão Paralela: Impacto do GIL

A eficiência diminui com o aumento de threads devido ao Global Interpreter Lock (GIL), limitando o paralelismo real em Python.

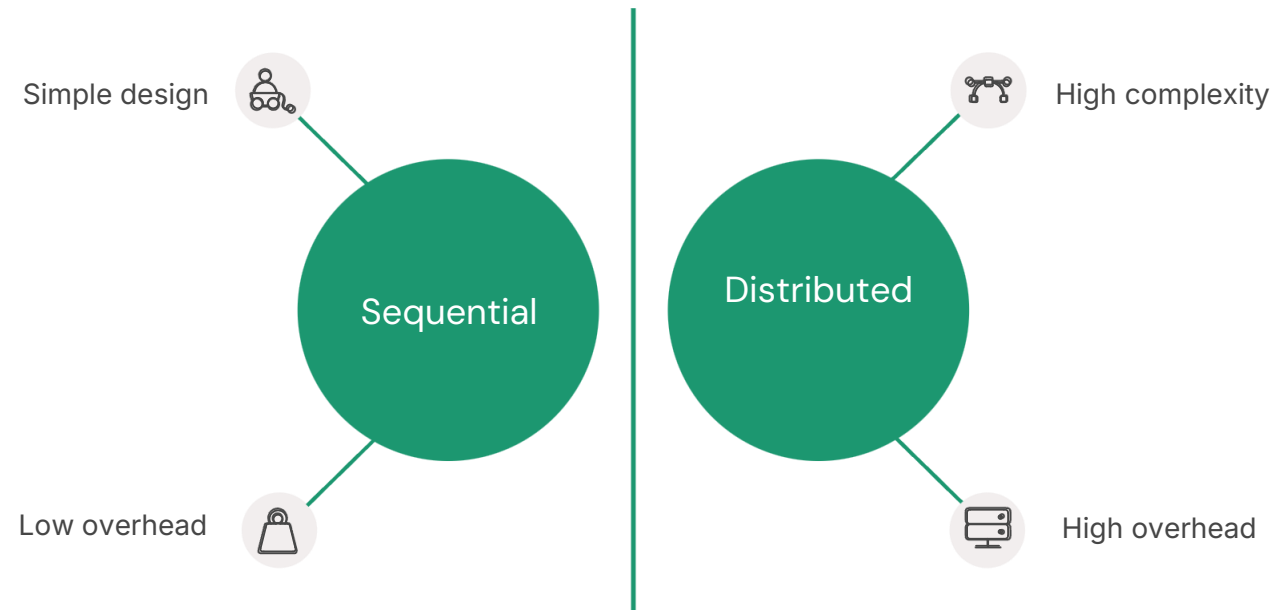
Melhor em Malhas Grandes

Problemas com malhas maiores demonstram maior eficiência, pois o tempo de computação supera o overhead de coordenação.

Versão Distribuída: Custo da Comunicação

A latência de comunicação entre os workers penaliza a eficiência, especialmente em problemas menores ou com muitos nós.

Principais Conclusões



Versão Sequencial

A abordagem mais simples, porém com desempenho limitado em problemas complexos ou de grande volume.



Versão Paralela

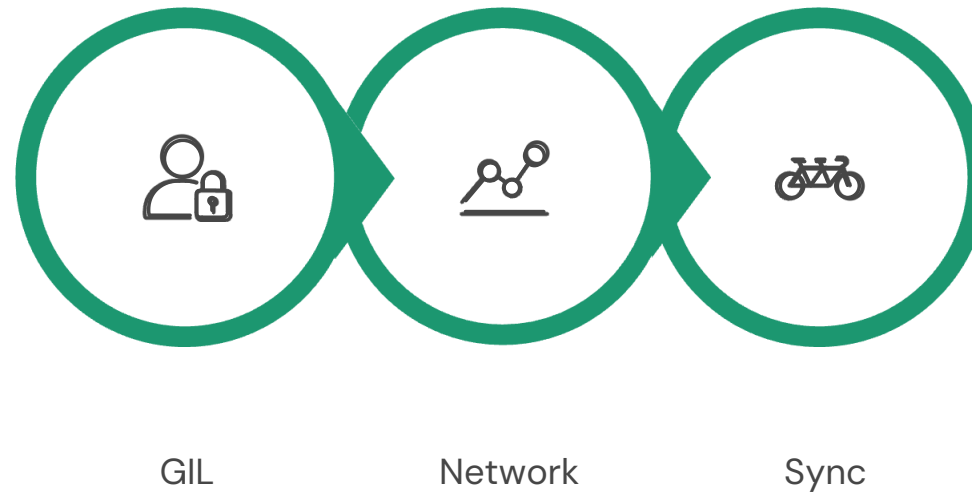
Oferece bons ganhos de desempenho, mas sua escalabilidade é restrita pelo Global Interpreter Lock (GIL) em Python.



Versão Distribuída

Torna-se vantajosa apenas para problemas muito grandes, onde os benefícios da escalabilidade superam o custo de comunicação.

Gargalos Encontrados



GIL (Global Interpreter Lock)

O Global Interpreter Lock (GIL) em Python restringe a execução de múltiplas threads de CPU simultaneamente, limitando o paralelismo real e impactando o desempenho em aplicações CPU-bound.

Latência e Serialização na Rede

A comunicação de dados e a serialização entre nós em sistemas distribuídos introduzem atrasos significativos e overhead, reduzindo a eficiência e escalabilidade.

Sincronização Custosa

A coordenação e sincronização de processos ou threads em ambientes paralelos e distribuídos podem gerar um overhead substancial, consumindo recursos e desacelerando a execução.

Problemas Técnicos e Soluções



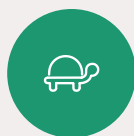
Interpretador Incorreto

O ambiente de desenvolvimento (VSCode) estava configurado para utilizar um interpretador Python diferente do esperado, causando inconsistências e falhas na execução.



Convergência Imediata

Atingia-se a convergência rapidamente devido a configurações inadequadas ou valores iniciais que limitavam a exploração do espaço de solução, resultando em resultados subótimos.



Baixo Speedup em Threads

Apesar do uso de múltiplas threads, o ganho de desempenho era marginal, principalmente devido às limitações impostas pelo Global Interpreter Lock (GIL) em Python.



Alta Latência Distribuída

A comunicação entre os componentes da versão distribuída apresentava alta latência, impactando negativamente a performance e a escalabilidade do sistema.

Melhorias de Curto Prazo



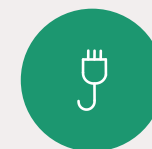
Multiprocessing

Substituição de threads por multiprocessing para contornar o Global Interpreter Lock (GIL) do Python, permitindo paralelismo real e melhor utilização de múltiplos núcleos de CPU.



Vetorização NumPy

Implementação de vetorização total utilizando NumPy para otimizar operações em arrays e eliminar loops Python lentos, resultando em ganhos significativos de desempenho computacional.



MPI (Message Passing Interface)

Substituição de sockets por MPI para otimizar a comunicação entre processos em ambientes distribuídos, reduzindo a latência e melhorando a escalabilidade do sistema.

Melhorias de Médio Prazo



Numba JIT Compilation

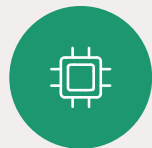
Compilação JIT (Just-In-Time) de loops Python usando Numba, convertendo-os em código de máquina otimizado em tempo de execução para ganhos significativos de performance, especialmente em operações numéricas e científicas.



Híbrido MPI + OpenMP

Combinação estratégica de MPI para comunicação entre nós de memória distribuída e OpenMP para paralelismo dentro de um nó de memória compartilhada, maximizando a eficiência em sistemas computacionais heterogêneos.

Melhorias de Longo Prazo



GPU com CUDA

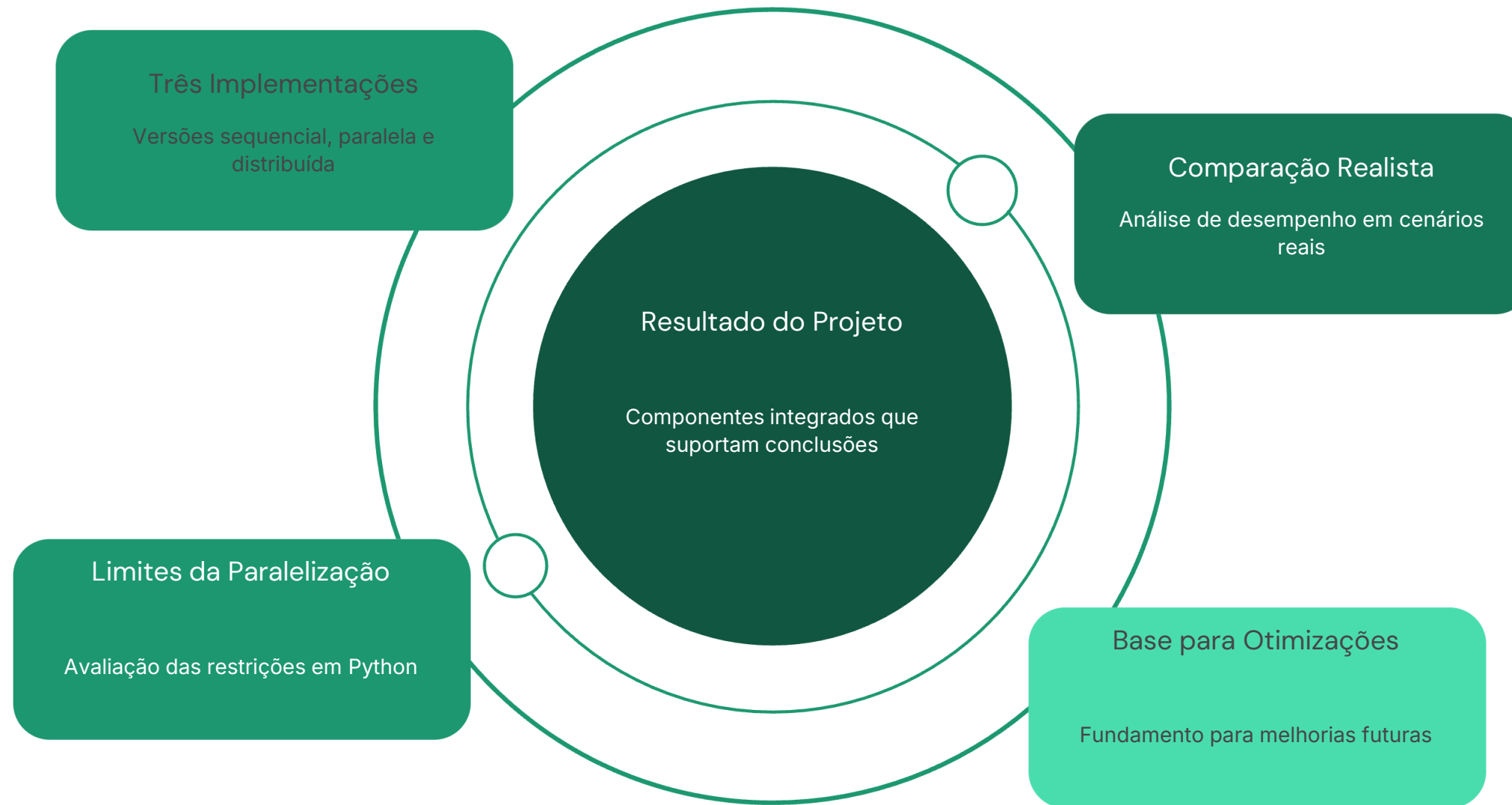
Implementação em GPU para máximo desempenho, aproveitando o poder de processamento paralelo para aceleração computacional.



Métodos Numéricos Avançados

Otimização de algoritmos com Multigrid, ADI e Jacobi otimizado para maior eficiência e precisão na resolução de problemas complexos.

O que o Trabalho Entregou



Obrigado



Filipe Antonio de Lima Nogueira



João Victor da Cruz Silvestre



Matheus Henrique Consoni
Mazanti

Este projeto é o resultado da dedicação e colaboração de uma equipe comprometida com a inovação e o sucesso.